DRI Domain Kit Manual

The gert Project

April 21, 2026

Contents

1 Introduction **1**
1.1 What This Manual Covers . . . 1
1.2 Audience and Prerequisites . . . 2
1.3 What is DRI? . . . 3
1.4 How gert.ops Implements DRI . . . 4
1.4.1 Step Types . . . 4
1.4.2 Metadata Fields . . . 4
1.4.3 Evidence Types . . . 5
1.4.4 Governance Integration . . . 5
1.5 Document Structure . . . 5

2 DRI Concepts **7**
2.1 The DRI Model . . . 7
2.2 Role Taxonomy . . . 7
2.2.1 dri Directly Responsible Individual . . . 8
2.2.2 approver . . . 8
2.2.3 change-manager . . . 9
2.2.4 incident-commander . . . 9
2.2.5 responder . . . 10
2.2.6 observer . . . 10
2.3 Role Assignment in Runbooks . . . 10
2.4 The Escalation Chain . . . 11
2.5 SLA Concepts . . . 12
2.5.1 Execution Window . . . 13
2.5.2 Maximum Duration . . . 13
2.5.3 Escalation Policy . . . 13
2.6 DRI Transfer . . . 14

3 Schema Reference **15**
3.1 Kit Declaration and `apiVersion` . . . 15
3.2 Runbook Metadata Fields . . . 16
3.2.1 `meta.roles` . . . 16
3.2.2 `meta.change-id` . . . 17
3.2.3 `meta.sla` . . . 17
3.3 Step Types . . . 17
3.3.1 `ops.cli` . . . 18
3.3.2 `ops.manual` . . . 19
3.3.3 `ops.approval` . . . 19
3.3.4 `ops.change-request` . . . 21
3.3.5 `ops.incident` . . . 22
3.4 Evidence Types . . . 23
3.4.1 `ops.evidence.screenshot` . . . 23
3.4.2 `ops.evidence.command-output` . . . 24
3.4.3 `ops.evidence.attestation` . . . 24

4 Authoring Guide **26**
4.1 Starting a New Runbook . . . 26
4.2 Declaring Roles . . . 27
4.3 Governance Configuration . . . 28
4.4 Using Approval Gates . . . 28
4.5 Using the Change-Request Wrapper . . . 29
4.6 Capturing Evidence . . . 31
4.7 Best Practices . . . 32
4.8 Complete Example: Production Deployment . . . 32

5 Governance **37**
5.1 Core Governance Hooks . . . 37
5.2 Role-Based Gating . . . 37
5.2.1 `requires_role` . . . 38
5.2.2 `requires_any_role` . . . 38
5.2.3 `requires_all_roles` . . . 38
5.3 Allowlist Enforcement in `ops.cli` Steps . . . 39
5.4 Environment Variable Blocking . . . 40
5.5 Output Redaction Patterns . . . 40

5.6 The Audit Trail . . . 41
5.7 Complete Governance Configuration Example . . . 42

6 Evidence Tracing **45**
6.1 The Evidence Model . . . 45
6.2 Evidence Record Structure . . . 46
6.3 Run Directory Structure . . . 47
6.4 Reading the Timeline Projection . . . 47
6.5 Evidence Bundles for Compliance . . . 48
6.6 SHA-256 Verification Workflow . . . 49
6.7 Reading Evidence from a Completed Run . . . 50

7 Change Request Workflow **52**
7.1 Lifecycle Overview . . . 52
7.2 Pre-Execution Phase . . . 52
7.2.1 Change-Manager Approval Gate . . . 52
7.2.2 Freeze Window Check . . . 53
7.3 Execution Phase . . . 53
7.3.1 Automatic Rollback Injection . . . 54
7.4 Post-Execution Phase . . . 54
7.5 Complete Example: Database Schema Migration . . . 54

8 Incident Response Workflow **61**
8.1 Incident Lifecycle . . . 61
8.2 Declaring an Incident . . . 62
8.3 Incident Commander Takeover . . . 62
8.4 Triage Steps . . . 63
8.5 Escalation When SLA Timeout Fires . . . 63
8.6 Complete Example: Service Degradation Incident . . . 63

9 Migration **71**
9.1 Migration Overview . . . 71
9.2 Core v1 v2 Migration (Summary) . . . 71
9.3 Adding the `gert.ops` Kit . . . 72
9.4 `gert.ops`-Specific Migration Patterns . . . 73
9.4.1 Old `manual ops.manual` . . . 73
9.4.2 Old Approval Steps `ops.approval` . . . 74

9.4.3 Bare `cli` Deployment Steps `ops.change-request` 75
9.4.4 Unstructured Incident Runbooks `ops.incident` 76
9.5 Migration Validator 77
9.6 Migration Validation Checklist 77

10 Reference **79**
10.1 Runbook Metadata Fields 80
10.2 `ops.cli` Field Reference 81
10.3 `ops.manual` Field Reference 81
10.4 `ops.approval` Field Reference 82
10.5 `ops.change-request` Field Reference 82
10.6 `ops.incident` Field Reference 83
10.7 Evidence Type Field Reference 83
10.7.1 Common Fields (all evidence types) 83
10.7.2 `ops.evidence.screenshot` 83
10.7.3 `ops.evidence.command-output` 83
10.7.4 `ops.evidence.attestation` 84
10.8 Role Semantics Reference 84
10.9 Environment Variables 84
10.10Error Codes 85

Chapter 1

Introduction

1.1 What This Manual Covers

This manual is the authoritative reference for the `gert.ops` Domain Kit—a specialized vocabulary layer for authoring operations runbooks within the gert v2 framework. The `gert.ops` Kit implements the DRI (Directly Responsible Individual) accountability model, providing step types, governance primitives, and evidence-capture patterns designed for SRE and DevOps teams running production operations, change management, and incident response.

The manual covers:

- The DRI accountability model and role taxonomy
- `gert.ops` step types: `ops.cli`, `ops.manual`, `ops.approval`, `ops.change-request`, `ops.incident`
- Role-based gating, approval workflows, and SLA timeout policies
- Evidence types: screenshots, command output, attestations with SHA256 verification
- Change request workflows: pre-execution approval, evidence capture, rollback injection
- Incident response workflows: declaration, triage, escalation, resolution
- Migration from v1 DRI runbooks to gert v2 + `gert.ops`
- Complete field reference for all `gert.ops` schema elements

This is *not* a general introduction to gert v2 or Domain Kits. For core gert concepts (runbook structure, core step types, the Domain Kit model), see the *gert v2 Design Document*. For guidance on developing your own Domain Kit, see the *Domain Kit Development Guide*.

1.2 Audience and Prerequisites

This manual is written for:

SRE and DevOps Engineers Writing runbooks for deployments, rollbacks, infrastructure changes, and service maintenance.

Operations Leads Designing change management processes, approval matrices, and escalation policies.

Incident Commanders Authoring incident response playbooks with triage steps and evidence requirements.

Change Managers Defining governance policies, approval gates, and audit trail requirements for compliance.

Prerequisites. You should be familiar with:

- **gert v2 core concepts** runbook structure, core step types (`cli`, `manual`, `tool`, `branch`, `iterate`), variables, and the execution model. See *gert v2 Design Document*, ğ1–ğ3.
- **Domain Kit concepts** what a Domain Kit is, how Kit-specific step types compile (lower) into core primitives, and the relationship between authoring vocabulary and runtime execution. See *gert v2 Design Document*, ğ4.
- **YAML syntax** `gert.ops` runbooks are authored in YAML. You do not need to know JSON Schema or Go.

If you are new to gert, start with the core documentation before reading this manual. The `gert.ops` Kit builds on core concepts and assumes you understand the foundation.

1.3 What is DRI?

DRI (Directly Responsible Individual) is an operational accountability model that designates exactly one person as accountable for a task, change, or incident at any given moment. The DRI is not necessarily the person *doing* the work—they are the person *responsible* for the outcome.

Origin. The DRI model originated at Apple and has been widely adopted in SRE and DevOps cultures. In operational contexts, DRI provides clarity during high-stress events: when an incident occurs or a change is being deployed, everyone knows who has authority to make decisions and who is accountable if things go wrong.

Why DRI Matters. Operations teams often face situations where multiple people are involved (a deployment might require an engineer, a database admin, and a manager's approval), but no one is clearly accountable. This creates confusion:

- Who decides whether to proceed with a risky step?
- Who is authorized to approve a rollback?
- Who escalates if a timeout is reached?
- Who signs off on the evidence after completion?

The DRI model resolves this by making accountability explicit. At every moment during a runbook execution, there is one DRI. If the DRI is unavailable, responsibility transfers to a designated backup via the escalation chain.

DRI in gert.ops. The `gert.ops` Domain Kit implements DRI accountability through:

- A role taxonomy: `dri`, `approver`, `change-manager`, `incident-commander`, `responder`, `observer`
- Step-level role requirements: `requires_role` field enforces that only authorized roles can execute a step
- Approval gates: `ops.approval` step pauses execution until specified roles sign off
- Evidence signatures: every evidence item records which role captured it

- SLA enforcement: timeouts trigger escalation to backup roles if the primary DRI is unresponsive

1.4 How gert.ops Implements DRI

The `gert.ops` Kit extends gert v2 core with five domain-specific step types and additional metadata fields. These additions compile down to core primitives at build time, so the gert runtime never sees `gert.ops`-specific vocabulary. This compilation model ensures that the core engine remains stable and domain-agnostic.

1.4.1 Step Types

`ops.cli` Extends core `cli` with audit logging, command allowlist enforcement, and environment variable blocking.

`ops.manual` Extends core `manual` with role requirements, SLA timeouts, and evidence type constraints.

`ops.approval` An approval gate: pauses execution until specified roles sign off. Compiles into a `manual` step with approval-check logic.

`ops.change-request` Wraps a sequence of steps in a change management context, recording change ID, approval timestamp, and automatic rollback step injection on failure.

`ops.incident` Wraps a sequence of steps in an incident response context, assigning incident ID, incident-commander role, and escalation chain.

1.4.2 Metadata Fields

`gert.ops` runbooks include additional metadata:

`meta.roles` A map of role names to identities (person, team, or `@on-call`). Example: `dri: alice@example.com`, `approver: ops-team`.

`meta.change-id` Optional reference to an external change management system (e.g., JIRA ticket, ServiceNow change record).

`meta.sla` Execution window and escalation policy. Defines the maximum allowed duration and who to notify on timeout.

1.4.3 Evidence Types

`gert.ops` defines three evidence types that extend core evidence capture:

`ops.evidence.screenshot` An image attachment with SHA256 hash, MIME type, and role signature.

`ops.evidence.command-output` Captured stdout/stderr from a `cli` step, with SHA256 and role signature.

`ops.evidence.attestation` Free-text evidence with timestamp and role signature (e.g., "I verified the database replica lag is under 100ms").

1.4.4 Governance Integration

`gert.ops` wires approval gates into gert core's governance hooks. When a `ops.approval` gate is reached:

1. The runtime pauses and emits a `governance/approvalRequired` trace event.
2. The runtime waits for approval decisions to be submitted via the `gert exec approve` command or API.
3. Once the minimum number of approvals from specified roles is met, execution resumes.
4. If the SLA timeout elapses before approvals are received, the runtime triggers the escalation policy (either fail the run or escalate to a backup approver).

1.5 Document Structure

The remaining chapters are organized as follows:

Chapter 1: DRI Concepts The DRI accountability model, role taxonomy, escalation chains, and SLA policies.

Chapter 2: Schema Reference The `gert.ops` Kit schema, step types, and metadata fields.

Chapter 3: Authoring Guide How to write a `gert.ops` runbook from scratch, with complete examples.

Chapter 4: Governance Role-based gating, allowlist enforcement, environment variable blocking, redaction, and the audit trail.

Chapter 5: Evidence and Tracing The evidence model, timeline projection, SHA256 verification, and compliance bundles.

Chapter 6: Change Request Workflow End-to-end walkthrough of a change request from pre-approval to closure.

Chapter 7: Incident Response Workflow End-to-end walkthrough of an incident from declaration to resolution.

Chapter 8: Migration Migrating v1 DRI runbooks to gert v2 + `gert.ops`.

Chapter 9: Reference Complete field reference tables for all `gert.ops` schema elements.

Each chapter is designed to be self-contained. If you are writing a deployment runbook, you can start with Chapter 3 (Authoring Guide) and refer to Chapter 2 (Schema Reference) and Chapter 9 (Field Reference) as needed. If you are designing an incident response playbook, start with Chapter 7. If you are migrating existing runbooks, start with Chapter 8.

Chapter 2

DRI Concepts

2.1 The DRI Model

The **Directly Responsible Individual** (DRI) model is a single-owner accountability pattern. At any moment during a runbook execution, exactly one person is the DRI: they have authority to make decisions, approve gates, and sign off on evidence. The DRI is accountable for the outcome—if a change fails, the DRI answers for it. If an incident escalates, the DRI owns the response.

Single Owner Invariant. A runbook execution has exactly one DRI at any given time. The DRI may change during the run (e.g., when an incident-commander takes over), but at no point are there zero DRIs or multiple conflicting DRIs.

DRI vs. Executor. The DRI is not always the person executing the steps. For example:

- A database engineer may run the migration steps, but the DRI is the lead engineer who approved the plan.
- During an incident, multiple responders may be troubleshooting, but the incident-commander is the DRI.

The DRI is accountable, not necessarily hands-on-keyboard.

2.2 Role Taxonomy

The `gert.ops` Kit defines six role types. Each role has specific permissions and responsibilities.

2.2.1 dri Directly Responsible Individual

Permissions:

- Execute all steps (unless a step requires a more specific role)
- Submit evidence
- Approve gates if listed as an approver
- Cancel the run

Responsibilities:

- Own the outcome of the runbook execution
- Make decisions when the runbook requires judgment calls
- Sign off on the final evidence bundle

Assignment: The DRI is declared in `meta.roles.dri`. If no DRI is declared, the person starting the run becomes the implicit DRI.

2.2.2 approver

Permissions:

- Sign off on `ops.approval` gates
- View the run status and evidence

Responsibilities:

- Review the evidence before signing off
- Reject approval if conditions are not met

Assignment: Approvers are listed in `meta.roles.approver` (may be a person, team, or on-call alias). Specific approval gates may override this with a gate-specific approver list.

2.2.3 change-manager

Permissions:

- Approve or reject the run before it starts
- View the final evidence bundle

Responsibilities:

- Ensure the change request is properly authorized
- Verify that the run is scheduled during an approved change window
- Sign off on the change record after completion

Assignment: Declared in `meta.roles.change-manager`. If a `ops.change-request` step exists, the change-manager must approve before execution begins.

2.2.4 incident-commander

Permissions:

- Take over as DRI when an incident is declared
- Assign tasks to responders
- Escalate or de-escalate the incident severity

Responsibilities:

- Coordinate the incident response
- Decide when to execute rollback steps
- Declare the incident resolved

Assignment: Declared in `meta.roles.incident-commander`. When an `ops.incident` step is executed, the incident-commander becomes the active DRI for the duration of the incident.

2.2.5 responder

Permissions:

- Execute steps that require the `responder` role
- Submit evidence
- Communicate with the incident-commander

Responsibilities:

- Execute assigned triage or mitigation tasks
- Report status to the incident-commander

Assignment: Declared in `meta.roles.responder` (may be a team or on-call alias).

2.2.6 observer

Permissions:

- View the run status and evidence in real-time
- Receive notifications when specific events occur

Responsibilities:

- None. Observers are read-only participants.

Assignment: Declared in `meta.roles.observer`. Observers are typically stakeholders, managers, or compliance auditors who need visibility but not control.

2.3 Role Assignment in Runbooks

Roles are assigned in two places:

Runbook-level role declarations. The `meta.roles` block declares the default mapping of role names to identities:

```
meta:
  roles:
    dri: alice@example.com
    approver: ops-team
    change-manager: change-board
    incident-commander: @on-call/sre
    responder: sre-team
    observer: [manager@example.com, compliance@example.com]
```

Step-level role requirements. Individual steps may require a specific role to execute:

```
steps:
  - id: migrate-database
    type: ops.cli
    requires_role: dri
    command: ./migrate.sh

  - id: verify-replication
    type: ops.manual
    requires_role: responder
    prompt: Check that all replicas are caught up
```

If a step has `requires_role`, only the person assigned to that role may execute the step. The runtime will block execution and emit an error if the current actor does not have the required role.

2.4 The Escalation Chain

If the primary DRI is unavailable (e.g., they go offline during an incident), responsibility must transfer to a backup. The `gert.ops` Kit supports escalation chains:

```
meta:
  roles:
    dri: alice@example.com
```

```
escalation:
  - primary: alice@example.com
    backup: bob@example.com
    timeout: 10m
  - primary: bob@example.com
    backup: @on-call/sre
    timeout: 5m
```

How escalation works.

1. The runtime starts with the primary DRI (alice).
2. If alice does not respond to an approval gate or manual step within 10 minutes, the runtime escalates to bob.
3. If bob does not respond within 5 minutes, the runtime escalates to the on-call SRE.
4. If the final backup does not respond, the run fails with a timeout error.

Escalation triggers. Escalation is triggered by:

- `ops.approval` timeout: the approval gate specifies a timeout, and no approver signs off before the deadline.
- `ops.manual` timeout: a manual step specifies a timeout, and the DRI does not mark it complete.
- SLA violation: the `meta.sla.max_duration` is exceeded, and the run is still in progress.

2.5 SLA Concepts

SLA (Service Level Agreement) policies define the execution window and timeout behavior for a runbook.

2.5.1 Execution Window

The execution window is the time period during which the runbook is allowed to run. If the run is started outside the window, it is rejected:

```yaml
meta:
  sla:
    execution_window:
      start: "2025-01-15T02:00:00Z"
      end: "2025-01-15T06:00:00Z"
```

This is useful for change management: a deployment runbook may only be executed during a scheduled maintenance window.

2.5.2 Maximum Duration

The maximum duration is the longest time the runbook is allowed to take:

```yaml
meta:
  sla:
    max_duration: 2h
```

If the run exceeds this duration, the runtime triggers the escalation policy (either fail the run or escalate to a backup DRI).

2.5.3 Escalation Policy

The escalation policy defines what happens when an SLA timeout is reached:

```yaml
meta:
  sla:
    on_timeout: escalate  # or: fail
```

`fail` Terminate the run and emit a `run/failed` event.

`escalate` Transfer DRI responsibility to the next person in the escalation chain.

2.6 DRI Transfer

DRI transfer occurs when the primary DRI hands off responsibility to another person. This is common during incident response when shifts change or when an escalation timeout fires.

Explicit transfer. The DRI may explicitly transfer ownership:

```
$ gert exec transfer --run-id abc123 --to bob@example.com
```

The runtime records the transfer in the trace:

```
{
  "type": "governance/driTransfer",
  "timestamp": "2025-01-15T03:45:00Z",
  "from": "alice@example.com",
  "to": "bob@example.com",
  "reason": "shift change"
}
```

Automatic transfer. Automatic transfer occurs when:

- An `ops.incident` step is executed and the incident-commander takes over as DRI.
- An escalation timeout fires and the backup DRI is activated.

Same-actor-cannot-approve invariant. When the DRI transfers, the new DRI cannot approve their own gates. If alice was the DRI and bob takes over, bob cannot approve a gate that requires the `approver` role if bob is both the DRI and the approver. This separation-of-duties rule prevents self-approval.

Chapter 3

Schema Reference

This chapter is the authoritative reference for every schema construct in the `gert.ops` Domain Kit. It covers the Kit declaration, all five `gert.ops` step types, runbook metadata fields, and the three evidence types. For the core gert v2 schema (step types `cli`, `manual`, `tool`, `branch`, `iterate`), refer to the *gert v2 Design Document*, ğ3.

3.1 Kit Declaration and `apiVersion`

A `gert.ops` runbook declares both the core schema version and the Kit in the `apiVersion` and `kit` fields at the top of the file.

```yaml
apiVersion: runbook/v2
kit: ops/v1

meta:
  name: my-ops-runbook
  kind: change
  description: "Short description of what this runbook does."
```

`apiVersion: runbook/v2` Required. Selects the gert v2 core schema. The `gert.ops` Kit is not compatible with `runbook/v1` runbooks.

`kit: ops/v1` Required for `gert.ops` runbooks. Activates the Kit compiler, making `ops.*` step types and metadata fields available. Omitting this field produces a validation error when any `ops.*` type is used.

3.2 Runbook Metadata Fields

The `meta` block in a `gert.ops` runbook extends the core metadata with Kit-specific fields. Core fields (`name`, `kind`, `description`, `inputs`, `governance`) continue to work as described in the core documentation.

3.2.1 `meta.roles`

Declares the named participants for this runbook execution. Each field accepts a string (a specific username or on-call alias) or a list of strings.

```yaml
meta:
  roles:
    dri: "@alice"
    approver:
      - "@bob"
      - "@change-board"
    change-manager: "@change-board"
    responder:
      - "@oncall-sre"
    incident-commander: "@ic-oncall"
    observer:
      - "@audit-team"
```

`dri` The Directly Responsible Individual for this run. Exactly one value. If omitted, the person starting the run becomes the implicit DRI.

`approver` One or more identities that may approve `ops.approval` gates. Gate-level `approvers` fields override this list.

`change-manager` The identity responsible for change authorization. Required when the runbook contains an `ops.change-request` step.

`responder` Identities that may execute `ops.manual` steps during an incident. Required when the runbook contains an `ops.incident` wrapper.

`incident-commander` The identity that assumes DRI during incident declaration. Defaults to `dri` if unset.

`observer` Read-only participants who receive run notifications and can view the evidence bundle but cannot take action.

3.2.2 `meta.change-id`

A reference to an external change ticket (e.g., ServiceNow, Jira) that authorized this runbook execution. Used for audit trail cross-referencing.

```
meta:
  change-id: "CHG0042817"
```

If the runbook contains an `ops.change-request` step, `meta.change-id` is populated automatically from the change request record after pre-execution approval is granted.

3.2.3 `meta.sla`

Defines SLA timeouts that apply at the runbook level. These are defaults; individual steps may override them.

```
meta:
  sla:
    approval-timeout: 30m
    execution-timeout: 4h
    escalation-target: "@sre-lead"
```

`approval-timeout` Maximum time to wait for a gate approval before the run is automatically escalated. Duration string (e.g., `15m`, `1h`).

`execution-timeout` Maximum total wall-clock time for the entire runbook run. If this duration elapses, the run is suspended and the DRI is notified.

`escalation-target` Identity notified when a timeout fires.

3.3 Step Types

The `gert.ops` Kit defines five step types. All five are lowered to core primitives at compile time. The runtime never executes `ops.*` nodes directly.

3.3.1 `ops.cli`

An audited command-line step. Extends the core `cli` type with an explicit command allowlist, argument-level redaction, and an automatic evidence capture option.

```yaml
- step:
    id: deploy_image
    type: ops.cli
    title: "Deploy container image"
    command: kubectl
    args:
      - rollout
      - restart
      - "deployment/{{ .service_name }}"
      - "-n"
      - "{{ .namespace }}"
    allowed_commands: [kubectl]
    redact:
      - pattern: "--token=[^ ]+"
        replace: "--token=[REDACTED]"
    capture:
      rollout_output: stdout
    evidence:
      auto: true
      label: "kubectl rollout output"
```

`command` The executable to run. Subject to the governance allowlist.

`args` Argument list. Template expressions are evaluated before execution.

`allowed_commands` Step-level allowlist override. Merges with the runbook-level `meta.governance.allowed_c`

`redact` List of RE2 pattern/replace pairs applied to stdout and stderr before capture and evidence writing.

`capture` Maps run variables to captured output streams.

`evidence.auto` When `true`, the step output is automatically captured as an `ops.evidence.command-output` record.

`evidence.label` Human-readable label for the auto-captured evidence record.

3.3.2 `ops.manual`

A manual procedure step requiring human action and evidence submission. Extends the core `manual` type with a role requirement and structured evidence specification.

```yaml
- step:
    id: verify_deployment
    type: ops.manual
    title: "Verify deployment is healthy"
    requires_role: dri
    instructions: |
      1. Open the Grafana dashboard for {{ .service_name }}.
      2. Confirm error rate < 0.1% over the last 5 minutes.
      3. Confirm p99 latency < 200ms.
      4. Take a screenshot of the dashboard.
    evidence:
      - type: ops.evidence.screenshot
        label: "Grafana dashboard post-deploy"
        required: true
      - type: ops.evidence.attestation
        label: "DRI sign-off"
        required: true
        prompt: "I confirm the service is healthy and the deployment is
↪   successful."
```

`requires_role` The role that must execute this step. One of: `dri`, `approver`, `change-manager`, `responder`, `incident-commander`.

`instructions` Free-form instructions shown to the executor. Template expressions are supported.

`evidence` List of evidence specifications. Each entry has `type`, `label`, `required`, and type-specific fields (see ğ3.4).

3.3.3 `ops.approval`

A blocking gate that requires explicit approval from one or more designated approvers before execution may continue. Lowers to a `manual` step with an approval-specific evidence record and an SLA timeout.

```yaml
- step:
    id: change_board_approval
    type: ops.approval
    title: "Change board sign-off"
    description: |
      Review the deployment plan and approve to proceed.
      Runbook: {{ .runbook_name }}
      Target environment: {{ .env }}
      Scheduled window: {{ .change_window }}
    approvers:
      - "@change-board"
      - "@sre-lead"
    requires_any: true
    timeout: 24h
    on_timeout: escalate
    escalation_target: "@vp-engineering"
    evidence:
      - type: ops.evidence.attestation
        label: "Change board approval"
        required: true
        prompt: "I approve this change for the scheduled window."
```

description Context shown to approvers in the approval request. Templates are evaluated when the gate is reached.

approvers List of identities that may sign this gate. Overrides `meta.roles.approver` for this specific gate.

requires_any When `true` (default), any single approver may sign. When `false`, *all* listed approvers must sign.

timeout Duration to wait for approval. Defaults to `meta.sla.approval-timeout` if not set.

on_timeout Action when timeout fires. `escalate` notifies the `escalation_target` and suspends the run; `abort` cancels the run.

escalation_target Identity notified on timeout. Overrides `meta.sla.escalation-target`.

3.3.4 `ops.change-request`

A wrapper step that groups a deployment or rollback sequence under a formal change request lifecycle. Provides pre-execution approval, rollback injection on failure, and automatic change record closure.

```yaml
- step:
    id: deploy_change
    type: ops.change-request
    title: "Deploy v2.4.1 to production"
    change-id: "{{ .change_ticket }}"
    risk: low
    rollback:
      runbook: rollback-service.runbook.yaml
      inputs:
        service_name: "{{ .service_name }}"
        target_version: "{{ .previous_version }}"
    steps:
      - step:
          id: pull_image
          type: ops.cli
          title: "Pull container image"
          command: docker
          args: [pull, "{{ .image_ref }}"]
          evidence:
            auto: true
      - step:
          id: apply_manifests
          type: ops.cli
          title: "Apply Kubernetes manifests"
          command: kubectl
          args: [apply, -f, "{{ .manifest_path }}"]
          evidence:
            auto: true
```

`change-id` Reference to the external change ticket. Populates `meta.change-id` if not already set.

`risk` Risk classification: `low`, `medium`, `high`, `emergency`. Used by the change-manager gate to determine required approvals.

`rollback` Rollback specification. If any step within `steps` fails, gert automatically initiates the specified rollback runbook.

`rollback.runbook` Path to the rollback runbook (relative to the current runbook file).

`rollback.inputs` Input bindings passed to the rollback runbook. Template expressions reference the current run's variables.

`steps` The change execution steps, executed in order.

3.3.5 `ops.incident`

A wrapper step that declares an operational incident and activates incident-response mode. All steps within the wrapper are executed under incident-commander authority, with mandatory evidence capture and automatic SLA escalation.

```
- step:
    id: declare_incident
    type: ops.incident
    title: "Service degradation  payment processing"
    severity: sev2
    incident-id: "INC-{{ .incident_ticket }}"
    commander: "@ic-oncall"
    sla:
      mitigation-timeout: 30m
      resolution-timeout: 4h
      escalation-target: "@vp-engineering"
    steps:
      - step:
          id: initial_triage
          type: ops.manual
          title: "Initial triage"
          requires_role: responder
          instructions: |
            Check error rates, latency, and recent deployments.
```

```
        evidence:
          - type: ops.evidence.attestation
            label: "Triage findings"
            required: true
            prompt: "Describe the symptoms and initial findings."
```

severity Incident severity: `sev1`, `sev2`, `sev3`, `sev4`. Severity `sev1` requires immediate IC takeover.

incident-id External incident ticket reference (e.g., PagerDuty, Opsgenie).

commander Identity that assumes DRI for the duration of the incident wrapper. Defaults to `meta.roles.incident-commander`.

sla.mitigation-timeout Time allowed to achieve initial mitigation.

sla.resolution-timeout Total time allowed for full resolution.

steps Incident response steps executed under incident-commander authority.

3.4 Evidence Types

`gert.ops` defines three evidence types. Evidence records are written to the run trace and included in the evidence bundle. All evidence records include a timestamp, the submitting user's identity, and their role at time of submission.

3.4.1 `ops.evidence.screenshot`

Captures a screenshot or image file submitted by the executor.

```
evidence:
  - type: ops.evidence.screenshot
    label: "Grafana dashboard  post-deploy"
    required: true
    description: "Screenshot showing error rate and latency for 5 minutes
↪   post-deploy."
```

label Short identifier for this evidence record in the bundle.

`required` When `true`, the step cannot complete without this evidence.

`description` Instructions shown to the submitter explaining what to capture.

At submission time, gert hashes the uploaded file (SHA-256), stores the binary under `.runbook/runs/<run-id>/attachments/<sha256>.bin`, and writes an `evidence/submitted` trace event with the hash and label.

3.4.2 `ops.evidence.command-output`

Captures the stdout/stderr of a CLI command as an evidence record. Produced automatically by `ops.cli` steps with `evidence.auto: true`, or explicitly declared.

```yaml
evidence:
  - type: ops.evidence.command-output
    label: "kubectl rollout status"
    required: true
    capture_from: rollout_output
```

`capture_from` Run variable containing the output to capture. Must be populated by a preceding `capture` block.

Command output is passed through the runbook's `redact` rules before storage.

3.4.3 `ops.evidence.attestation`

A structured sign-off where the executor explicitly affirms a statement. Used for DRI sign-offs, approvals, and change-manager authorizations.

```yaml
evidence:
  - type: ops.evidence.attestation
    label: "DRI deployment sign-off"
    required: true
    prompt: "I confirm this deployment is complete and the service is healthy."
```

`prompt` The exact statement the executor affirms. Displayed verbatim to the submitter before they confirm.

The attestation record stores the submitter's identity, their role, the prompt text, and the timestamp. It cannot be submitted by a user who does not hold the step's `requires_role`.

Evidence Record Written to Trace:

```yaml
# Trace event: evidence/submitted
event_id: "4a7b2c91-..."
kind: evidence/submitted
timestamp: "2026-05-14T03:22:11Z"
payload:
  step_id: "verify_deployment"
  evidence_label: "DRI deployment sign-off"
  evidence_type: "ops.evidence.attestation"
  submitted_by: "@alice"
  role: "dri"
  prompt: "I confirm this deployment is complete and the service is healthy."
  sha256: "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
```

Chapter 4

Authoring Guide

This chapter teaches you to write a `gert.ops` runbook from scratch. It walks through role declaration, approval gates, change-request wrapping, and evidence capture, ending with a complete, realistic deployment runbook you can use as a template.

4.1 Starting a New Runbook

Every `gert.ops` runbook begins with three declarations: the schema version, the Kit, and the metadata block.

```
apiVersion: runbook/v2
kit: ops/v1

meta:
  name: deploy-api-service
  kind: change
  description: >
    Deploy a new version of the API service to production.
    Requires change-board approval before execution.
  inputs:
    service_name:
      from: prompt
      description: "Service identifier (e.g., api-gateway)"
    image_tag:
      from: prompt
      description: "Docker image tag to deploy (e.g., v2.4.1)"
    change_ticket:
```

```
      from: prompt
      description: "Change request ticket number (e.g., CHG0042817)"
    previous_image_tag:
      from: prompt
      description: "Previous image tag, used for rollback"
```

Choosing a `kind`. Use `change` for planned deployments and infrastructure modifications, `incident` for incident-response runbooks, and `operational` for routine operational procedures (e.g., on-call runbooks, diagnostic playbooks).

4.2 Declaring Roles

Declare all participants in `meta.roles`. Be explicit: role declarations are enforced at runtime. A step with `requires_role: dri` will be blocked if the executing user does not hold the `dri` role.

```
meta:
  roles:
    dri: "@alice"
    approver:
      - "@bob"
      - "@change-board"
    change-manager: "@change-board"
    observer:
      - "@audit-log-team"
  sla:
    approval-timeout: 24h
    escalation-target: "@sre-lead"
```

Rule of thumb. Start with the DRI (who is accountable?), then the approver (who must sign off?), and the change-manager (who authorized the change window?). Add observers for audit requirements. If you are writing an incident runbook, also declare `responder` and `incident-commander`.

4.3 Governance Configuration

Declare the command allowlist and output redaction rules in `meta.governance`. For a deployment runbook, this typically covers the tools you use:

```yaml
meta:
  governance:
    allowed_commands: [kubectl, docker, curl, jq, helm]
    denied_commands: [rm, dd, shutdown]
    deny_env_vars:
      - "*_SECRET*"
      - "*_TOKEN"
      - "AWS_SECRET_ACCESS_KEY"
    redact:
      - pattern: "(--password=)[^ ]+"
        replace: "$1[REDACTED]"
      - pattern: "Bearer [A-Za-z0-9._-]+"
        replace: "Bearer [REDACTED]"
```

Every `ops.cli` step in the runbook inherits these rules. A step may additionally declare its own `allowed_commands` list, which is intersected with the runbook-level list.

4.4 Using Approval Gates

When to use an approval gate. Use `ops.approval` whenever execution must pause for a human decision:

- Before making a change to a production environment.
- After evidence has been gathered and a reviewer needs to sign off before continuing.
- At any point where policy requires documented authorization.

When *not* to use an approval gate. Do not use approval gates for informational pauses (use `ops.manual` without evidence instead). Do not chain multiple gates on the same step sequence unless they represent distinct approval authorities (e.g., a technical lead *and* a change manager are different roles).

```yaml
flow:
  - step:
      id: request_approval
      type: ops.approval
      title: "Change board approval required"
      description: |
        Deployment: {{ .service_name }}  {{ .image_tag }}
        Change ticket: {{ .change_ticket }}
        Planned window: {{ .change_window }}
      approvers:
        - "@change-board"
      requires_any: true
      timeout: 24h
      on_timeout: escalate
      escalation_target: "@vp-engineering"
      evidence:
        - type: ops.evidence.attestation
          label: "Change board sign-off"
          required: true
          prompt: "I approve this deployment for the scheduled change window."
```

Timeout behavior. When the timeout fires:

- With `on_timeout: escalate`: The run is suspended. The `escalation_target` receives a notification. The run can be resumed once approval is granted or the gate is bypassed by the DRI with documented justification.
- With `on_timeout: abort`: The run is cancelled. A trace event is written. No further steps execute.

4.5 Using the Change-Request Wrapper

Wrap your deployment or rollback execution steps in `ops.change-request` to activate the change management lifecycle. This provides automatic pre-execution approval from the change-manager, rollback injection on failure, and change record closure.

```yaml
  - step:
      id: execute_change
```

```yaml
type: ops.change-request
title: "Deploy {{ .service_name }} {{ .image_tag }}"
change-id: "{{ .change_ticket }}"
risk: medium
rollback:
  runbook: ./rollback-api-service.runbook.yaml
  inputs:
    service_name: "{{ .service_name }}"
    target_image_tag: "{{ .previous_image_tag }}"
steps:
  - step:
      id: apply_deployment
      type: ops.cli
      title: "Apply deployment manifest"
      command: kubectl
      args: [set, image, "deployment/{{ .service_name }}",
             "app={{ .image_tag }}", "-n", production]
      evidence:
        auto: true
        label: "kubectl set image output"
  - step:
      id: wait_rollout
      type: ops.cli
      title: "Wait for rollout to complete"
      command: kubectl
      args: [rollout, status, "deployment/{{ .service_name }}",
             "-n", production, "--timeout=5m"]
      evidence:
        auto: true
        label: "rollout status output"
```

Key point: The `ops.change-request` wrapper does not execute the change automatically. The change-manager must first approve the pre-execution gate (populated from `meta.roles.change-manager`). Only after that approval do the `steps` execute.

4.6 Capturing Evidence

Choose the evidence type that best matches the proof you need:

`ops.evidence.command-output` Use when the proof is the output of a command (e.g., `kubectl rollout status`, `curl /healthz`). Enable `evidence.auto: true` on `ops.cli` steps to capture automatically.

`ops.evidence.screenshot` Use when the proof is a visual dashboard state (e.g., Grafana, Datadog) that cannot be captured programmatically. Requires the executor to upload a file.

`ops.evidence.attestation` Use for formal sign-offs: DRI acknowledgements, approvals, and change-manager authorizations. The executor affirms a specific statement.

Evidence on manual steps.

```yaml
  - step:
      id: health_check
      type: ops.manual
      title: "Confirm service health"
      requires_role: dri
      instructions: |
        1. Open Grafana: https://grafana.internal/d/api-service
        2. Confirm error rate < 0.1% for 5 minutes post-deploy.
        3. Confirm p99 latency < 200ms.
        4. Take a screenshot of the dashboard.
      evidence:
        - type: ops.evidence.screenshot
          label: "Grafana health dashboard"
          required: true
          description: "Dashboard showing error rate and latency, 5 min
↪  post-deploy."
        - type: ops.evidence.attestation
          label: "DRI health sign-off"
          required: true
          prompt: "I confirm the service is healthy and the deployment is
↪  successful."
```

4.7 Best Practices

One DRI, always. Every runbook must have a clear DRI. If the runbook can be run by anyone, use `dri: "@oncall-sre"` to assign the on-call engineer.

Gate before irreversible actions. Place an `ops.approval` gate before every step sequence that modifies production state and cannot be trivially undone.

Always specify rollback. Every `ops.change-request` wrapper must have a `rollback` specification. Do not skip this even for "low risk" changes.

Capture evidence at the point of action. Use `evidence.auto: true` on `ops.cli` steps rather than a separate manual evidence step after the fact. The evidence should capture the actual state, not a post-hoc reconstruction.

Keep instructions human-readable. The `instructions` field in `ops.manual` steps will be read under pressure. Use numbered lists and concrete URLs. Avoid pronouns and ambiguous references.

Use `meta.change-id`. Always link to the external change ticket. This allows compliance teams to cross-reference the gert evidence bundle with the change management system.

Test the rollback runbook. Run the rollback runbook in a non-production environment before scheduling the change window. An untested rollback is not a rollback.

4.8 Complete Example: Production Deployment

The following is a complete, production-ready deployment runbook demonstrating all major `gert.ops` constructs.

```
apiVersion: runbook/v2
kit: ops/v1

meta:
  name: deploy-api-service
  kind: change
  description: >
```

```
    Deploy a new version of the API service to production.
    Requires change-board approval before execution begins.
  inputs:
    service_name:
      from: prompt
      description: "Service identifier (e.g., api-gateway)"
    image_tag:
      from: prompt
      description: "Docker image tag to deploy (e.g., v2.4.1)"
    change_ticket:
      from: prompt
      description: "Change ticket number (e.g., CHG0042817)"
    previous_image_tag:
      from: prompt
      description: "Previous image tag for rollback"
    namespace:
      from: prompt
      description: "Kubernetes namespace (e.g., production)"
      default: "production"
  roles:
    dri: "@alice"
    approver:
      - "@change-board"
    change-manager: "@change-board"
    observer:
      - "@audit-team"
  sla:
    approval-timeout: 24h
    escalation-target: "@sre-lead"
  change-id: "{{ .change_ticket }}"
  governance:
    allowed_commands: [kubectl, curl, jq]
    deny_env_vars: ["*_SECRET*", "*_TOKEN"]

flow:
  # - Gate 1: Change board approval ---------------------
  - step:
```

```yaml
    id: change_board_approval
    type: ops.approval
    title: "Change board approval"
    description: |
      Deployment request:
        Service:  {{ .service_name }}
        Image:    {{ .image_tag }}
        Ticket:   {{ .change_ticket }}
        Namespace: {{ .namespace }}
    approvers: ["@change-board"]
    requires_any: true
    timeout: 24h
    on_timeout: escalate
    escalation_target: "@sre-lead"
    evidence:
      - type: ops.evidence.attestation
        label: "Change board approval"
        required: true
        prompt: >
          I approve deployment of {{ .image_tag }} to {{ .namespace }}
          under change ticket {{ .change_ticket }}.

# - Change request wrapper -------------------------
- step:
    id: execute_deployment
    type: ops.change-request
    title: "Deploy {{ .service_name }} {{ .image_tag }}"
    change-id: "{{ .change_ticket }}"
    risk: medium
    rollback:
      runbook: ./rollback-api-service.runbook.yaml
      inputs:
        service_name: "{{ .service_name }}"
        target_image_tag: "{{ .previous_image_tag }}"
        namespace: "{{ .namespace }}"
    steps:
      - step:
```

```
          id: set_image
          type: ops.cli
          title: "Update deployment image"
          command: kubectl
          args:
            - set
            - image
            - "deployment/{{ .service_name }}"
            - "app={{ .image_tag }}"
            - "-n"
            - "{{ .namespace }}"
          evidence:
            auto: true
            label: "kubectl set image"

      - step:
          id: wait_rollout
          type: ops.cli
          title: "Wait for rollout to complete"
          command: kubectl
          args:
            - rollout
            - status
            - "deployment/{{ .service_name }}"
            - "-n"
            - "{{ .namespace }}"
            - "--timeout=5m"
          capture:
            rollout_status: stdout
          evidence:
            auto: true
            label: "rollout status"

# - Post-deploy health check ------------------------
- step:
    id: health_check
    type: ops.manual
```

```
      title: "Verify deployment health"
      requires_role: dri
      instructions: |
        1. Open Grafana: https://grafana.internal/d/{{ .service_name }}
        2. Confirm error rate < 0.1% over 5 min post-deploy.
        3. Confirm p99 latency < 200ms.
        4. Confirm no new alerts in PagerDuty.
        5. Take a screenshot of the Grafana dashboard.
      evidence:
        - type: ops.evidence.screenshot
          label: "Grafana health dashboard"
          required: true
          description: "Dashboard showing error rate and latency, 5 min
↪  post-deploy."
        - type: ops.evidence.attestation
          label: "DRI health sign-off"
          required: true
          prompt: >
            I confirm {{ .service_name }} {{ .image_tag }} is healthy in
↪  production.
            Error rate is within SLO and latency is acceptable.
```

Chapter 5

Governance

This chapter explains how `gert.ops` wires into gert's core governance layer and extends it with role-based gating, Kit-specific policy primitives, and audit trail generation. For the core governance model (allowlists, denylists, env-var blocking, output redaction), see the *gert v2 Design Document*, ğ11. This chapter covers the `gert.ops`-specific extensions to that model.

5.1 Core Governance Hooks

The gert core governance layer evaluates policy at two points in the execution pipeline:

Pre-fork checkpoint Runs immediately before any subprocess is started. Checks the command allowlist/denylist and strips blocked environment variables.

Post-capture checkpoint Runs after a step completes and output is captured. Applies output redaction rules before writing to trace or run variables.

`gert.ops` adds a third checkpoint:

Pre-step role checkpoint Runs before any step executes. For steps with `requires_role`, verifies that the current executor holds the required role. If the role check fails, the step is blocked and a `governance/roleBlocked` trace event is written.

5.2 Role-Based Gating

`gert.ops` steps support three role-gating modifiers. These are evaluated at the pre-step role checkpoint.

5.2.1 `requires_role`

The executing user must hold exactly the specified role.

```yaml
- step:
    id: approve_rollback
    type: ops.manual
    title: "Authorize rollback"
    requires_role: change-manager
    instructions: |
      Review the rollback plan and confirm authorization.
```

5.2.2 `requires_any_role`

The executing user must hold at least one of the listed roles.

```yaml
- step:
    id: triage_check
    type: ops.manual
    title: "Initial triage"
    requires_any_role:
      - dri
      - incident-commander
    instructions: |
      Perform initial triage and document findings.
```

5.2.3 `requires_all_roles`

The executing user must hold all of the listed roles simultaneously. Rare in practice; used when a single person must be both DRI and change-manager (e.g., emergency change procedures).

```yaml
- step:
    id: emergency_authorize
    type: ops.manual
    title: "Emergency change authorization"
    requires_all_roles:
      - dri
```

```
      - change-manager
    instructions: |
      As both DRI and change manager, authorize this emergency change.
```

Role inheritance. The `incident-commander` role inherits all permissions of `dri`. During an `ops.incident` wrapper execution, the declared commander may execute any step gated on `requires_role: dri`.

5.3 Allowlist Enforcement in `ops.cli` Steps

Command allowlists in `gert.ops` follow the core model but are evaluated hierarchically:

1. **Runbook-level:** `meta.governance.allowed_commands` applies to all steps unless overridden.
2. **Step-level:** `ops.cli.allowed_commands` intersected with the runbook-level list. A command not in the runbook-level list cannot be unblocked at the step level.
3. **Denylist:** `meta.governance.denied_commands` always applied first, regardless of allowlist.

```
meta:
  governance:
    allowed_commands: [kubectl, helm, curl, jq, aws]
    denied_commands: [rm, dd, shutdown, reboot, curl]  # curl denied globally

flow:
  - step:
      id: check_pods
      type: ops.cli
      title: "List running pods"
      command: kubectl
      args: [get, pods, -n, production]
      # Step-level allowlist narrows to kubectl only; curl remains denied.
      allowed_commands: [kubectl]
```

When a command is blocked, gert writes a `governance/commandBlocked` trace event and fails the step with exit code 126.

5.4 Environment Variable Blocking

The `deny_env_vars` list accepts glob patterns. Before any subprocess fork, the full process environment is scanned and matching variable names are stripped.

```yaml
meta:
  governance:
    deny_env_vars:
      - "*_SECRET*"          # AWS_SECRET_ACCESS_KEY, DB_SECRET_PASSWORD, etc.
      - "*_TOKEN"            # GITHUB_TOKEN, VAULT_TOKEN, etc.
      - "AWS_*"              # All AWS credential vars
      - "KUBECONFIG"         # Prevent implicit cluster targeting
      - "DOCKER_HOST"        # Prevent Docker daemon redirection
```

Trace event. When variables are stripped, a `governance/envVarBlocked` event is written. The event records the *names* of blocked variables, never the values:

```yaml
# Trace event: governance/envVarBlocked
kind: governance/envVarBlocked
payload:
  step_id: "deploy_image"
  var_names: ["AWS_SECRET_ACCESS_KEY", "GITHUB_TOKEN"]
  reason: "matched deny_env_vars pattern"
```

5.5 Output Redaction Patterns

Output redaction applies to the captured stdout and stderr of `ops.cli` steps before the output is stored in run variables or written to evidence records.

```yaml
meta:
  governance:
    redact:
      # Bearer tokens in HTTP responses
      - pattern: "Bearer [A-Za-z0-9._\\-]+"
        replace: "Bearer [REDACTED]"
      # Password in connection strings
      - pattern: ":[^@:]+@"
```

```
    replace: ":[REDACTED]@"
  # AWS access keys (20 uppercase alphanum chars)
  - pattern: "(?:AKIA|ASIA)[A-Z0-9]{16}"
    replace: "[AWS-KEY-REDACTED]"
  # kubectl --token flag
  - pattern: "(--token=)[^ ]+"
    replace: "$1[REDACTED]"
```

Patterns use RE2 syntax. The `replace` field may reference capture groups with `$1`, `$2`, etc. Rules are applied in declaration order. After all rules are applied, the redacted output is stored; the raw output is discarded and never written to the trace.

5.6 The Audit Trail

The `gert.ops` audit trail is the sequence of governance events in the run trace. The following events are written by the governance layer:

`governance/roleBlocked` A user attempted to execute a step requiring a role they do not hold. Fields: `step_id`, `user`, `required_role`, `actual_roles`.

`governance/commandBlocked` A command was blocked by the allowlist or denylist. Fields: `step_id`, `command`, `reason`.

`governance/envVarBlocked` Environment variables were stripped before subprocess fork. Fields: `step_id`, `var_names`.

`governance/approvalGranted` An approval gate was signed. Fields: `step_id`, `approved_by`, `role`, `timestamp`.

`governance/approvalRejected` An approval gate was rejected. Fields: `step_id`, `rejected_by`, `role`, `reason`.

`governance/approvalTimeout` An approval gate timed out. Fields: `step_id`, `action` (`escalate` or `abort`).

`evidence/submitted` An evidence record was submitted. Fields: `step_id`, `evidence_label`, `evidence_type`, `submitted_by`, `role`, `sha256`.

These events, combined with the core `step/started` and `step/completed` events, form a tamper-evident audit log. The audit trail can be projected from the trace using `gert kit audit-report <run-id>`.

5.7 Complete Governance Configuration Example

```yaml
apiVersion: runbook/v2
kit: ops/v1

meta:
  name: infrastructure-change
  kind: change
  roles:
    dri: "@ops-lead"
    approver:
      - "@infra-change-board"
    change-manager: "@infra-change-board"
  sla:
    approval-timeout: 8h
    escalation-target: "@vp-ops"
  governance:
    allowed_commands:
      - terraform
      - aws
      - kubectl
      - jq
      - curl
    denied_commands:
      - rm
      - dd
      - shutdown
      - reboot
    deny_env_vars:
      - "*_SECRET*"
      - "*_TOKEN"
      - "AWS_SECRET_ACCESS_KEY"
```

```
      - "AWS_SESSION_TOKEN"
      - "TF_VAR_*"      # Block all Terraform variable overrides
    redact:
      - pattern: "(password\\s*=\\s*)[^\\s]+"
        replace: "$1[REDACTED]"
      - pattern: "(?:AKIA|ASIA)[A-Z0-9]{16}"
        replace: "[AWS-KEY-REDACTED]"
      - pattern: "Bearer [A-Za-z0-9._\\-]+"
        replace: "Bearer [REDACTED]"

flow:
  - step:
      id: infra_approval
      type: ops.approval
      title: "Infrastructure change approval"
      description: |
        Terraform plan for {{ .environment }} environment.
        Risk level: {{ .risk_level }}
        Ticket: {{ .change_ticket }}
      approvers: ["@infra-change-board"]
      requires_any: true
      timeout: 8h
      on_timeout: escalate
      evidence:
        - type: ops.evidence.attestation
          label: "Infra change board approval"
          required: true
          prompt: "I approve this infrastructure change for {{ .environment }}."

  - step:
      id: apply_terraform
      type: ops.change-request
      title: "Apply Terraform plan"
      change-id: "{{ .change_ticket }}"
      risk: "{{ .risk_level }}"
      rollback:
        runbook: ./rollback-terraform.runbook.yaml
```

```
        inputs:
          environment: "{{ .environment }}"
      steps:
        - step:
            id: tf_apply
            type: ops.cli
            title: "Terraform apply"
            command: terraform
            args: [apply, -auto-approve, "{{ .plan_file }}"]
            allowed_commands: [terraform]
            evidence:
              auto: true
              label: "terraform apply output"

  - step:
      id: dri_sign_off
      type: ops.manual
      title: "DRI post-change sign-off"
      requires_role: dri
      instructions: |
        Verify all Terraform resources are in the expected state.
        Check CloudWatch/monitoring for anomalies post-apply.
      evidence:
        - type: ops.evidence.attestation
          label: "DRI post-change confirmation"
          required: true
          prompt: "I confirm the infrastructure change was applied successfully
↪  with no anomalies."
```

Chapter 6

Evidence Tracing

This chapter describes the evidence capture model used by `gert.ops`: what is captured, when, how it is stored, and how to verify and read the evidence bundle from a completed run. The underlying trace format is defined by the gert v2 core (*gert v2 Design Document*, ğ12); this chapter focuses on the `ops.*`-layer semantics layered on top of it.

6.1 The Evidence Model

In `gert.ops`, evidence is the durable proof that a step was executed correctly by the right person, at the right time, with the right outcome. Evidence is not an audit log entry (those are written by the core); evidence is an artifact explicitly submitted or captured to prove a specific claim.

What gets captured.

- **Command output** (from `ops.cli` steps with `evidence.auto: true`): The redacted stdout/stderr of the command, the exit code, and the command invocation.
- **Screenshots** (from `ops.evidence.screenshot`): Binary files uploaded by the executor, stored as content-addressed attachments.
- **Attestations** (from `ops.evidence.attestation`): A structured record containing the prompt text, the submitter's identity and role, and a timestamp.
- **Approval records** (from `ops.approval`): Attestations submitted by approvers at gate steps.

What is *not* evidence. Governance events (`governance/commandBlocked`, `governance/envVarBlocked`) are written to the trace but are not evidence records. They are part of the audit log. Similarly, `step/started` and `step/completed` events are structural trace events, not evidence. Evidence records are things explicitly submitted by a human or captured from meaningful command output.

6.2 Evidence Record Structure

Every evidence submission writes an `evidence/submitted` trace event. The event payload has the following structure:

```yaml
# Trace event: evidence/submitted
event_id: "4a7b2c91-f3e8-4d2a-9c10-b5e1234567ab"
sequence: 14
kind: evidence/submitted
timestamp: "2026-05-14T03:22:11.842Z"
run_id: "20260514T032155-b1c3f9"
path: ["execute_deployment", "wait_rollout"]
payload:
  step_id: "wait_rollout"
  evidence_label: "rollout status output"
  evidence_type: "ops.evidence.command-output"
  submitted_by: "@alice"
  role: "dri"
  content_inline: "deployment.apps/api-service successfully rolled out\n"
  sha256: "a3f9c14b2e8d7f0612345678abcdef9012345678abcdef9012345678abcdef90"
  redacted: true
  command: "kubectl rollout status deployment/api-service -n production
↪  --timeout=5m"
  exit_code: 0
```

`step_id` The stable ID of the step that produced this evidence.

`evidence_label` The human-readable label declared in the runbook.

`evidence_type` One of `ops.evidence.command-output`, `ops.evidence.screenshot`, or `ops.evidence.attestation`.

`submitted_by` The identity of the person who submitted or triggered this evidence record.

`role` The role held by `submitted_by` at submission time.

`content_inline` Present for small text payloads (command output, attestation text). Absent for binary attachments; use `sha256` to retrieve.

`sha256` SHA-256 hash of the evidence payload. For binary attachments, this is the hash of the file stored in `.runbook/runs/<run-id>/attachments/`. For text evidence, this is the hash of the UTF-8 content.

`redacted` `true` if the content passed through the redaction pipeline before storage.

`command` Present for `command-output` evidence. The exact command invocation (redacted).

`exit_code` Present for `command-output` evidence.

6.3 Run Directory Structure

All run artifacts, including evidence, are stored under the run directory:

```
.runbook/runs/<run-id>/
  trace.jsonl           # Append-only event log (all events including evidence)
  snapshots/            # Per-step state checkpoints
  attachments/          # Binary evidence files (content-addressed by SHA-256)
    a3f9c14b2e8d7f06...bin   # Screenshot, binary output, etc.
  run.yaml              # Completion manifest
  evidence-bundle.json  # Projected evidence summary (written on completion)
  annotations.jsonl     # Operator notes
```

The `evidence-bundle.json` is a projected view over `trace.jsonl` containing only `evidence/submitted` events, sorted chronologically, with attachment references resolved. It is computed by `gert kit evidence-bundle <run-id>`.

6.4 Reading the Timeline Projection

The timeline projection shows all significant events in a run in chronological order. Generate it with:

```
$ gert kit timeline <run-id>
```

Example output for the deployment runbook from Chapter 3:

```
03:21:55Z  run/started          @alice (dri)
03:21:55Z  step/started         change_board_approval
03:22:00Z  governance/approvalGranted  @bob (approver)
03:22:00Z  evidence/submitted   change_board_approval / "Change board sign-off"
03:22:01Z  step/completed       change_board_approval  [approved]
03:22:02Z  step/started         execute_deployment
03:22:02Z  step/started         set_image
03:22:04Z  evidence/submitted   set_image / "kubectl set image"
03:22:04Z  step/completed       set_image  [exit:0]
03:22:05Z  step/started         wait_rollout
03:22:11Z  evidence/submitted   wait_rollout / "rollout status output"
03:22:11Z  step/completed       wait_rollout  [exit:0]
03:22:12Z  step/completed       execute_deployment  [change-closed:CHG0042817]
03:22:13Z  step/started         health_check
03:22:45Z  evidence/submitted   health_check / "Grafana health dashboard"
03:22:52Z  evidence/submitted   health_check / "DRI health sign-off"
03:22:52Z  step/completed       health_check
03:22:52Z  run/completed        @alice (dri)  [outcome:resolved]
```

6.5 Evidence Bundles for Compliance

For compliance review, generate the evidence bundle:

```
$ gert kit evidence-bundle <run-id>
```

The bundle is a JSON document containing:

```
{
  "run_id": "20260514T032155-b1c3f9",
  "runbook": "deploy-api-service",
  "started_at": "2026-05-14T03:21:55Z",
  "completed_at": "2026-05-14T03:22:52Z",
  "dri": "@alice",
```

```
  "change_id": "CHG0042817",
  "outcome": "resolved",
  "evidence": [
    {
      "step_id": "change_board_approval",
      "label": "Change board sign-off",
      "type": "ops.evidence.attestation",
      "submitted_by": "@bob",
      "role": "approver",
      "timestamp": "2026-05-14T03:22:00Z",
      "sha256": "e3b0c44298fc1c...",
      "prompt": "I approve deployment of v2.4.1..."
    },
    {
      "step_id": "wait_rollout",
      "label": "rollout status output",
      "type": "ops.evidence.command-output",
      "submitted_by": "@alice",
      "role": "dri",
      "timestamp": "2026-05-14T03:22:11Z",
      "sha256": "a3f9c14b2e8d7f06...",
      "command": "kubectl rollout status ...",
      "exit_code": 0
    }
  ],
  "bundle_sha256": "9f86d081884c7d659a2feaa0c55ad015..."
}
```


The `bundle_sha256` field is the SHA-256 of the entire JSON document (excluding the field itself), enabling tamper detection.

6.6 SHA-256 Verification Workflow

To verify an evidence bundle has not been tampered with:

```
# 1. Regenerate the bundle from the raw trace
$ gert kit evidence-bundle <run-id> --output=bundle.json
```

```
# 2. Verify individual attachment hashes
$ gert kit verify-attachments <run-id>
Verifying attachments for run 20260514T032155-b1c3f9:
  [OK] a3f9c14b2e8d7f06...bin  (rollout status output)
  [OK] 7f83b1657279c1f7...bin  (Grafana health dashboard)
All 2 attachments verified.

# 3. Verify the bundle SHA-256
$ gert kit verify-bundle <run-id>
Bundle SHA-256: 9f86d081884c7d659a2feaa0c55ad015...
Status: VERIFIED
```

Verification will fail with a non-zero exit code and a `TAMPERED` status if any trace event has been modified or any attachment file has changed since the run completed.

6.7 Reading Evidence from a Completed Run

```
# List all evidence records in a run
$ gert kit evidence list <run-id>
STEP                    LABEL                       TYPE          SUBMITTED-BY
change_board_approval   Change board sign-off      attestation   @bob
set_image               kubectl set image           cmd-output    @alice
wait_rollout            rollout status output       cmd-output    @alice
health_check            Grafana health dashboard    screenshot    @alice
health_check            DRI health sign-off         attestation   @alice

# Show a specific evidence record
$ gert kit evidence show <run-id> --step=wait_rollout --label="rollout status output"
Type:         ops.evidence.command-output
Step:         wait_rollout
Label:        rollout status output
Submitted by: @alice (dri)
Timestamp:    2026-05-14T03:22:11.842Z
SHA-256:      a3f9c14b2e8d7f0612345678abcdef90...
Command:      kubectl rollout status deployment/api-service -n production --timeout=5m
```

```
Exit code:    0
Redacted:     true
Content:
  deployment.apps/api-service successfully rolled out

# Export a screenshot attachment
$ gert kit evidence export <run-id> \
    --step=health_check \
    --label="Grafana health dashboard" \
    --output=grafana-screenshot.png
Exported: grafana-screenshot.png (SHA-256: 7f83b165...)
```

Chapter 7

Change Request Workflow

This chapter walks through the complete lifecycle of an `ops.change-request` execution, from the moment the runbook is started to change record closure. It includes a detailed example of a database schema migration with a rollback path.

7.1 Lifecycle Overview

A change request execution has three phases:

Pre-execution The change-manager reviews and approves the request. Freeze window checks are performed. The change record is opened in the external system.

Execution The DRI-guided steps run inside the `ops.change-request` wrapper. Evidence is captured at each step. Approval gates may pause execution.

Post-execution The DRI signs off. The change record is closed. The evidence bundle is written and available for compliance review.

If any step in the execution phase fails, the rollback sequence is automatically initiated before the run terminates.

7.2 Pre-Execution Phase

7.2.1 Change-Manager Approval Gate

When a runbook starts that contains an `ops.change-request` step, gert evaluates the pre-execution requirements before running any steps. The change-manager from `meta.roles.change-manager` must approve before execution begins.

The approval request is presented to the change-manager with:

- The runbook name, kind, and description.
- All declared inputs (with sensitive values redacted per governance rules).
- The `change-id` and `risk` level.
- The names of steps within the wrapper and any rollback specification.

7.2.2 Freeze Window Check

If the gert server is configured with a change freeze calendar, gert checks whether the current time falls within a freeze window before presenting the approval request. If a freeze window is active:

- Changes with `risk: low` or `risk: medium` are blocked.
- Changes with `risk: high` require an additional `ops.approval` gate explicitly authorizing emergency execution.
- Changes with `risk: emergency` bypass the freeze check.

7.3 Execution Phase

After pre-execution approval is granted, the steps inside the `ops.change-request` wrapper execute in declared order. The following rules apply during execution:

- Steps execute sequentially. Parallel execution is not supported within a change request wrapper.
- The change request is considered *open* from the moment the first inner step starts until either the wrapper completes successfully or the rollback completes.
- If a step fails (non-zero exit code, timeout, or explicit failure), gert pauses execution and presents the DRI with two options: **retry** (re-execute the failed step) or **rollback** (abandon the change and execute the rollback runbook).
- Evidence from steps within the wrapper is captured and tagged with the change request ID.

7.3.1 Automatic Rollback Injection

If the DRI chooses rollback (or if an unrecoverable error occurs), gert:

1. Writes a `change/rollbackInitiated` trace event with the failing step ID and reason.
2. Invokes the runbook specified in `rollback.runbook` with the inputs from `rollback.inputs`.
3. Waits for the rollback runbook to complete.
4. Writes a `change/rollbackCompleted` or `change/rollbackFailed` event.
5. Closes the current run with outcome `rolled-back`.

7.4 Post-Execution Phase

After all inner steps complete successfully, the change request wrapper:

1. Writes a `change/closed` trace event with the change ID, outcome, and DRI.
2. If the gert server is configured with an external change management integration, it updates the ticket (e.g., ServiceNow, Jira) with the run ID, outcome, and evidence bundle URL.
3. Returns control to the outer runbook. Steps after the `ops.change-request` wrapper execute normally (e.g., post-deploy health checks).

7.5 Complete Example: Database Schema Migration

The following is an annotated runbook for a database schema migration. It demonstrates the full change request lifecycle, including pre-execution approval, DRI-guided execution, evidence capture, and rollback specification.

```
apiVersion: runbook/v2
kit: ops/v1

meta:
  name: migrate-user-schema
  kind: change
```

```
  description: >
    Apply Flyway migration V42__add_user_preferences.sql to the production
    PostgreSQL database. Requires change-board approval and DRI sign-off.
  inputs:
    change_ticket:
      from: prompt
      description: "Change ticket (e.g., CHG0042819)"
    db_host:
      from: prompt
      description: "Database host (e.g., postgres.prod.internal)"
    db_name:
      from: prompt
      description: "Database name (e.g., users_db)"
    migration_version:
      from: prompt
      description: "Flyway migration version (e.g., V42)"
  roles:
    dri: "@db-lead"
    approver: ["@change-board"]
    change-manager: "@change-board"
    observer: ["@audit-team"]
  sla:
    approval-timeout: 24h
    escalation-target: "@vp-engineering"
  governance:
    allowed_commands: [flyway, psql, pg_dump, kubectl]
    deny_env_vars: ["*_SECRET*", "*_PASSWORD", "PGPASSWORD"]
    redact:
      - pattern: "(password=)[^\\s]+"
        replace: "$1[REDACTED]"

flow:
  # - Step 1: Back up the schema before migration --------------
  - step:
      id: pre_migration_backup
      type: ops.cli
      title: "Take pre-migration schema backup"
```

```
      command: pg_dump
      args:
        - "--schema-only"
        - "-h"
        - "{{ .db_host }}"
        - "-d"
        - "{{ .db_name }}"
        - "-f"
        - "pre-migration-schema.sql"
      capture:
        dump_output: stderr
      evidence:
        auto: true
        label: "pre-migration schema dump"

  # - Step 2: DRI reviews the migration script ---------------
  - step:
      id: review_migration
      type: ops.manual
      title: "DRI reviews migration script"
      requires_role: dri
      instructions: |
        Review the migration script before proceeding:
          cat migrations/{{ .migration_version }}__add_user_preferences.sql

        Confirm:
        1. The migration is non-destructive (no DROP TABLE, no column removal).
        2. The migration is backward compatible with the current application.
        3. An undo migration script exists at:
           migrations/undo/{{ .migration_version
↪   }}__rollback_user_preferences.sql
      evidence:
        - type: ops.evidence.attestation
          label: "DRI migration review"
          required: true
          prompt: >
            I have reviewed migration {{ .migration_version }} and confirm it is
```

```
          non-destructive and backward compatible.

# - Step 3: Change board approval --------------------
- step:
    id: change_board_approval
    type: ops.approval
    title: "Change board approval for schema migration"
    description: |
      Database schema migration request:
        Database: {{ .db_name }} on {{ .db_host }}
        Migration: {{ .migration_version }}__add_user_preferences.sql
        Ticket: {{ .change_ticket }}
        Risk: medium (additive schema change, backward compatible)
        Rollback: Flyway undo migration available
    approvers: ["@change-board"]
    requires_any: true
    timeout: 24h
    on_timeout: escalate
    evidence:
      - type: ops.evidence.attestation
        label: "Change board migration approval"
        required: true
        prompt: >
          I approve migration {{ .migration_version }} for {{ .db_name }}
          under change ticket {{ .change_ticket }}.

# - Step 4: Execute migration with rollback protection ----------
- step:
    id: execute_migration
    type: ops.change-request
    title: "Apply Flyway migration {{ .migration_version }}"
    change-id: "{{ .change_ticket }}"
    risk: medium
    rollback:
      runbook: ./rollback-db-migration.runbook.yaml
      inputs:
        db_host: "{{ .db_host }}"
```

```
      db_name: "{{ .db_name }}"
      migration_version: "{{ .migration_version }}"
  steps:
    # 4a. Validate migration checksums
    - step:
        id: flyway_validate
        type: ops.cli
        title: "Validate Flyway migration checksums"
        command: flyway
        args:
          - validate
          - "-url=jdbc:postgresql://{{ .db_host }}/{{ .db_name }}"
          - "-locations=filesystem:migrations"
        capture:
          validate_output: stdout
        evidence:
          auto: true
          label: "Flyway validate output"

    # 4b. Run the migration
    - step:
        id: flyway_migrate
        type: ops.cli
        title: "Apply migration"
        command: flyway
        args:
          - migrate
          - "-url=jdbc:postgresql://{{ .db_host }}/{{ .db_name }}"
          - "-locations=filesystem:migrations"
          - "-target={{ .migration_version }}"
        capture:
          migrate_output: stdout
        evidence:
          auto: true
          label: "Flyway migrate output"

    # 4c. Verify migration status
```

```
      - step:
          id: flyway_info
          type: ops.cli
          title: "Verify applied migrations"
          command: flyway
          args:
            - info
            - "-url=jdbc:postgresql://{{ .db_host }}/{{ .db_name }}"
          capture:
            info_output: stdout
          evidence:
            auto: true
            label: "Flyway info post-migration"

  # - Step 5: Post-migration verification -----------------
  - step:
      id: post_migration_check
      type: ops.manual
      title: "Verify application health after migration"
      requires_role: dri
      instructions: |
        1. Check application error logs: no DB-related errors in the last 5
↪  minutes.
        2. Run the application's health endpoint:
           curl https://api.prod.internal/healthz/db
        3. Confirm the new columns exist:
           psql -h {{ .db_host }} -d {{ .db_name }} \
             -c "SELECT column_name FROM information_schema.columns \
                 WHERE table_name = 'users' AND column_name LIKE
↪  '%preferences%';"
        4. Take a screenshot of the health check output.
      evidence:
        - type: ops.evidence.screenshot
          label: "Application health post-migration"
          required: true
          description: "Screenshot of health endpoint and schema verification."
        - type: ops.evidence.attestation
```

```
          label: "DRI post-migration sign-off"
          required: true
          prompt: >
            I confirm migration {{ .migration_version }} was applied
↪  successfully.
            The application is healthy and the new columns are present.
```

Chapter 8

Incident Response Workflow

This chapter covers the incident response lifecycle as implemented by the `ops.incident` wrapper. It walks through declaring an incident, commander takeover, triage, escalation, and resolution, concluding with a complete annotated example.

8.1 Incident Lifecycle

`gert.ops` models the incident lifecycle as six stages:

Detect An alert fires or a human observes symptoms. The on-call engineer is paged.

Declare The on-call engineer starts the incident runbook. The `ops.incident` wrapper activates, declaring the incident and assigning an Incident Commander.

Triage Responders gather data, run diagnostics, and determine root cause or impact scope. Each triage step requires evidence submission.

Mitigate Immediate steps to reduce customer impact (e.g., restart a service, roll back a deploy, apply a hotfix). Mitigation is tracked against the SLA timeout.

Resolve The root cause is addressed and the service is fully restored. The Incident Commander signs off on resolution.

Review Post-incident review (PIR). The evidence bundle from the run is the primary artifact for the PIR meeting. This stage is outside the runbook scope.

8.2 Declaring an Incident

Starting a runbook that contains an `ops.incident` wrapper immediately writes an `incident/declared` trace event and transitions the run into incident mode. In incident mode:

- All `ops.manual` steps with `requires_role: responder` are unlocked for any user listed in `meta.roles.responder`.
- SLA timers start immediately. The `mitigation-timeout` timer tracks time to first mitigation. The `resolution-timeout` timer tracks total incident duration.
- The declared `commander` assumes DRI authority for all steps within the `ops.incident` wrapper, even if a different DRI is declared at the runbook level.

8.3 Incident Commander Takeover

The Incident Commander (IC) may take over at any point during the run. Takeover is supported in two ways:

Declared takeover. The IC is declared in `ops.incident.commander`. When the wrapper starts, the IC immediately holds DRI authority.

Runtime takeover. During a live run, the IC can execute:

```
$ gert incident take-command <run-id> --as=@ic-oncall
```

This writes an `incident/commanderTakeover` trace event and reassigns DRI authority. The previous DRI is notified and transitions to `observer` status for subsequent steps.

DRI transfer back. After the incident is resolved, the IC can transfer DRI back to the original owner:

```
$ gert incident transfer-command <run-id> --to=@alice
```

8.4 Triage Steps

Triage steps use `ops.manual` with `requires_role: responder`. Each triage step should:

- Provide specific, actionable instructions (dashboards, runbook links, check commands).
- Require an attestation evidence record summarizing the findings.
- Use `ops.cli` sub-steps (or direct `ops.cli` steps) for diagnostic commands, capturing output as evidence.

Branch on triage outcome. Use the core `branch` step type within the incident wrapper to route to different mitigation steps based on triage findings.

8.5 Escalation When SLA Timeout Fires

When the `mitigation-timeout` fires, gert:

1. Writes an `incident/slaBreached` trace event with the timeout type and elapsed duration.
2. Notifies the `escalation-target` with the run ID, current step, and elapsed time.
3. The run continues. The escalation is informational; it does not pause or abort the run.

When the `resolution-timeout` fires, gert:

1. Writes an `incident/slaBreached` trace event.
2. Notifies the `escalation-target`.
3. Suspends the run and requires IC acknowledgement to continue.

8.6 Complete Example: Service Degradation Incident

The following runbook covers a service degradation incident from initial page to resolution.

```yaml
apiVersion: runbook/v2
kit: ops/v1

meta:
  name: service-degradation-response
  kind: incident
  description: >
    Incident response playbook for service degradation alerts.
    Covers triage, mitigation (rollback or restart), and resolution.
  inputs:
    service_name:
      from: prompt
      description: "Affected service (e.g., payment-processor)"
    incident_ticket:
      from: prompt
      description: "Incident ticket ID (e.g., INC-20240514-001)"
    alert_description:
      from: prompt
      description: "Brief description of the alert that triggered this runbook"
  roles:
    dri: "@oncall-sre"
    responder:
      - "@oncall-sre"
      - "@oncall-backend"
    incident-commander: "@ic-oncall"
    observer:
      - "@sre-team"
      - "@management-alerts"
  sla:
    mitigation-timeout: 30m
    resolution-timeout: 4h
    escalation-target: "@vp-engineering"
  governance:
    allowed_commands: [kubectl, curl, jq, pg_isready, redis-cli, stern]
    deny_env_vars: ["*_SECRET*", "*_TOKEN", "*_KEY"]

flow:
```

```
  # - Incident declaration and lifecycle wrapper ---------------
  - step:
      id: incident_response
      type: ops.incident
      title: "Service degradation: {{ .service_name }}"
      severity: sev2
      incident-id: "{{ .incident_ticket }}"
      commander: "@ic-oncall"
      sla:
        mitigation-timeout: 30m
        resolution-timeout: 4h
        escalation-target: "@vp-engineering"
      steps:
        # - Stage 1: Initial triage ----------------------
        - step:
            id: initial_triage
            type: ops.manual
            title: "Initial triage"
            requires_role: responder
            instructions: |
              Alert: {{ .alert_description }}
              Service: {{ .service_name }}

              Check the following:
              1. Error rate: https://grafana.internal/d/{{ .service_name
↪  }}/errors
              2. Latency: https://grafana.internal/d/{{ .service_name }}/latency
              3. Recent deployments in the last 2 hours:
                 kubectl rollout history deployment/{{ .service_name }} -n
↪  production
              4. Pod health:
                 kubectl get pods -n production -l app={{ .service_name }}
              5. Recent logs (last 100 lines):
                 stern {{ .service_name }} -n production --tail=100
            evidence:
              - type: ops.evidence.attestation
                label: "Initial triage findings"
```

```
        required: true
        prompt: >
          Describe the symptoms: error rate, latency, pod status, and
          whether a recent deployment is likely the cause.

  # - Stage 2: Diagnostic commands --------------------
  - step:
      id: check_pod_status
      type: ops.cli
      title: "Check pod status"
      command: kubectl
      args:
        - get
        - pods
        - "-n"
        - production
        - "-l"
        - "app={{ .service_name }}"
        - "-o"
        - wide
      capture:
        pod_status: stdout
      evidence:
        auto: true
        label: "Pod status"

  - step:
      id: check_recent_events
      type: ops.cli
      title: "Check recent Kubernetes events"
      command: kubectl
      args:
        - get
        - events
        - "-n"
        - production
        - "--sort-by=.lastTimestamp"
```

```
            - "--field-selector"
            - "involvedObject.name={{ .service_name }}"
          capture:
            k8s_events: stdout
          evidence:
            auto: true
            label: "Recent Kubernetes events"

      # - Stage 3: Mitigation decision -------------------
      - step:
          id: mitigation_decision
          type: ops.manual
          title: "Select mitigation strategy"
          requires_role: incident-commander
          instructions: |
            Pod status: {{ .pod_status }}
            Events: {{ .k8s_events }}

            Based on the triage findings, choose the mitigation strategy:
            - If a recent deployment is the cause: select ROLLBACK
            - If pods are crash-looping with no recent deploy: select RESTART
            - If the issue is external (DB, upstream): select ESCALATE

            Document your reasoning before selecting.
          evidence:
            - type: ops.evidence.attestation
              label: "IC mitigation decision"
              required: true
              prompt: >
                State the root cause hypothesis and chosen mitigation
↪  strategy.
                Justify why this strategy was selected.

      # - Stage 4a: Rollback mitigation ------------------
      # (This branch is executed when the IC determines rollback is needed.
      #  In practice, wrap in a branch step on the mitigation_decision
↪  choice.)
```

```yaml
    - step:
        id: rollback_deployment
        type: ops.cli
        title: "Rollback to previous deployment"
        command: kubectl
        args:
          - rollout
          - undo
          - "deployment/{{ .service_name }}"
          - "-n"
          - production
        capture:
          rollback_output: stdout
        evidence:
          auto: true
          label: "kubectl rollout undo output"

    - step:
        id: wait_rollback
        type: ops.cli
        title: "Wait for rollback to complete"
        command: kubectl
        args:
          - rollout
          - status
          - "deployment/{{ .service_name }}"
          - "-n"
          - production
          - "--timeout=5m"
        evidence:
          auto: true
          label: "rollback rollout status"

    # - Stage 5: Post-mitigation health check --------------
    - step:
        id: mitigation_health_check
        type: ops.cli
```

```
            title: "Check service health endpoint"
            command: curl
            args:
              - "-sf"
              - "https://{{ .service_name }}.prod.internal/healthz"
              - "-o"
              - /dev/null
              - "-w"
              - "%{http_code}"
            capture:
              health_code: stdout
            evidence:
              auto: true
              label: "health endpoint response code"

        # - Stage 6: Resolution sign-off --------------------
        - step:
            id: resolution_sign_off
            type: ops.manual
            title: "IC resolution sign-off"
            requires_role: incident-commander
            instructions: |
              Confirm all of the following before signing off:
              1. Error rate has returned to normal (< 0.1%).
              2. Latency is within SLO (p99 < 200ms).
              3. No crash-looping pods.
              4. Health endpoint returns HTTP 200.

              Dashboard: https://grafana.internal/d/{{ .service_name }}
            evidence:
              - type: ops.evidence.screenshot
                label: "Post-mitigation Grafana dashboard"
                required: true
                description: "Dashboard showing error rate and latency, 10 min
↪  post-mitigation."
              - type: ops.evidence.attestation
                label: "IC resolution declaration"
```

```
      required: true
      prompt: >
        I declare incident {{ .incident_ticket }} resolved.
        {{ .service_name }} is healthy. Mitigation was successful.
        Root cause: [state root cause].
        Action items for PIR: [list items or 'none'].
```

Chapter 9

Migration

This chapter provides a complete migration guide for teams moving from v1 DRI runbooks to gert v2 + `gert.ops`. It covers the automated migration tool, manual migration patterns, and a validation checklist.

9.1 Migration Overview

Migrating to `gert.ops` involves two independent concerns:

Core v1 v2 migration Schema field promotions, step type renames, and new required fields. Handled by `gert migrate`. See the *gert v2 Design Document*, ğ11 for the core migration reference.

`gert.ops` Kit adoption Adding the Kit declaration, replacing informal manual steps with typed `ops.*` steps, declaring roles, adding approval gates, wrapping deployment sequences in `ops.change-request`, and specifying evidence requirements. This is a semantic upgrade; it requires author judgment and cannot be fully automated.

You must complete the core v1 v2 migration before applying `gert.ops`-specific patterns.

9.2 Core v1 v2 Migration (Summary)

Run the automated migrator on your existing v1 runbooks:

```
$ gert migrate --to-v2 my-deploy.runbook.yaml
```

```
Migrating: my-deploy.runbook.yaml
  [REWRITE] apiVersion: runbook/v1  runbook/v2
  [REWRITE] step type: collector  manual
  [REWRITE] step type: router  end
  [PROMOTE] meta.vars  top-level vars
  [ADD]     id field derived from meta.name
Output: my-deploy.runbook.yaml (modified in place)
Backup: my-deploy.runbook.yaml.v1.bak
```

The migrator rewrites the file in place and saves a backup with the `.v1.bak` suffix. After migration, validate the file:

```
$ gert validate my-deploy.runbook.yaml
my-deploy.runbook.yaml: valid (runbook/v2)
```

For a full list of v1 v2 breaking changes, refer to the *gert v2 Design Document*, ğ11.

9.3 Adding the `gert.ops` Kit

After v2 migration, add the Kit declaration and role metadata:

```yaml
# Before (v2, no kit):
apiVersion: runbook/v2
meta:
  name: deploy-service
  kind: change

# After (v2 + gert.ops):
apiVersion: runbook/v2
kit: ops/v1
meta:
  name: deploy-service
  kind: change
  roles:
    dri: "@ops-lead"
    approver: ["@change-board"]
    change-manager: "@change-board"
```

```
  sla:
    approval-timeout: 24h
    escalation-target: "@sre-lead"
```

9.4 gert.ops-Specific Migration Patterns

9.4.1 Old `manual` → `ops.manual`

V2 core `manual` steps that require human action and evidence should be upgraded to `ops.manual` to gain role enforcement and structured evidence capture.

```
# Before (v2 core manual):
- step:
    id: verify_health
    type: manual
    title: "Verify service health"
    instructions: |
      Check the dashboard and confirm the service is healthy.
    required_evidence:
      - kind: text
        name: health_notes

# After (ops.manual with role + typed evidence):
- step:
    id: verify_health
    type: ops.manual
    title: "Verify service health"
    requires_role: dri
    instructions: |
      Check https://grafana.internal/d/my-service.
      Confirm error rate < 0.1% and p99 latency < 200ms.
    evidence:
      - type: ops.evidence.screenshot
        label: "Health dashboard"
        required: true
        description: "Grafana dashboard showing error rate and latency."
      - type: ops.evidence.attestation
```

```
        label: "DRI health sign-off"
        required: true
        prompt: "I confirm the service is healthy."
```

9.4.2 Old Approval Steps `ops.approval`

V1 runbooks often used `manual` or `collector` steps as informal approval gates. Migrate these to `ops.approval` to get timeout enforcement, formal approver lists, and audit events.

```
# Before (v1 informal approval):
- step:
    id: get_approval
    type: collector
    title: "Get manager approval"
    instructions: |
      Ask your manager to approve this change before continuing.
    required_evidence:
      - kind: text
        name: approval_note

# After (ops.approval with timeout and escalation):
- step:
    id: get_approval
    type: ops.approval
    title: "Manager approval required"
    description: |
      Deployment: {{ .service_name }} {{ .image_tag }}
      Ticket: {{ .change_ticket }}
    approvers: ["@change-board"]
    requires_any: true
    timeout: 8h
    on_timeout: escalate
    escalation_target: "@sre-lead"
    evidence:
      - type: ops.evidence.attestation
        label: "Manager approval"
        required: true
        prompt: "I approve this deployment."
```

9.4.3 Bare `cli` Deployment Steps `ops.change-request`

V1 deployment runbooks typically used bare `cli` steps without change management lifecycle or rollback injection. Wrap these in `ops.change-request`.

```yaml
# Before (v1 bare cli steps):
- step:
    id: deploy
    type: cli
    command: kubectl
    args: [set, image, "deployment/api", "app={{ .image_tag }}"]

# After (wrapped in ops.change-request):
- step:
    id: execute_deployment
    type: ops.change-request
    title: "Deploy {{ .service_name }} {{ .image_tag }}"
    change-id: "{{ .change_ticket }}"
    risk: medium
    rollback:
      runbook: ./rollback.runbook.yaml
      inputs:
        service_name: "{{ .service_name }}"
        target_image_tag: "{{ .previous_image_tag }}"
    steps:
      - step:
          id: deploy
          type: ops.cli
          title: "Update deployment image"
          command: kubectl
          args: [set, image, "deployment/{{ .service_name }}", "app={{
↪  .image_tag }}"]
          evidence:
            auto: true
```

9.4.4 Unstructured Incident Runbooks `ops.incident`

V1 incident runbooks were often flat sequences of manual steps with no lifecycle tracking. Wrap the triage and mitigation steps in `ops.incident` to activate SLA timers, commander takeover, and incident-mode evidence requirements.

```yaml
# Before (flat manual steps):
- step:
    id: triage
    type: manual
    title: "Triage the incident"
    instructions: "Check dashboards and logs."

# After (ops.incident wrapper):
- step:
    id: incident_response
    type: ops.incident
    title: "Incident response"
    severity: sev2
    incident-id: "{{ .incident_ticket }}"
    commander: "@ic-oncall"
    sla:
      mitigation-timeout: 30m
      resolution-timeout: 4h
      escalation-target: "@vp-engineering"
    steps:
      - step:
          id: triage
          type: ops.manual
          title: "Triage the incident"
          requires_role: responder
          instructions: "Check dashboards and logs."
          evidence:
            - type: ops.evidence.attestation
              label: "Triage findings"
              required: true
              prompt: "Describe the symptoms and initial hypothesis."
```

9.5 Migration Validator

After migration, use the Kit validator to check for `gert.ops`-specific issues:

```
$ gert kit validate --migration my-deploy.runbook.yaml

Validating: my-deploy.runbook.yaml (kit: ops/v1)

[WARN]  No ops.approval gate before ops.change-request wrapper (step: execute_deployment).
        Consider adding an approval gate before production changes.

[WARN]  ops.change-request at step execute_deployment has no rollback specification.
        Production changes should always declare a rollback runbook.

[ERROR] ops.manual step verify_health has no requires_role declaration.
        ops.manual steps must declare a role to enforce accountability.

[INFO]  meta.roles.dri is not set. The run initiator will become implicit DRI.
        Explicit role declaration is recommended for production runbooks.

1 error, 2 warnings, 1 info.
```

The `-migration` flag enables additional warnings for patterns common in migrated runbooks that do not yet take full advantage of `gert.ops` features.

9.6 Migration Validation Checklist

Use this checklist to verify a complete `gert.ops` migration:

#	Check	Status
1	`apiVersion: runbook/v2` declared	☐
2	`kit: ops/v1` declared	☐
3	`gert validate` passes with no errors	☐
4	`gert kit validate` passes with no errors	☐
5	`meta.roles.dri` explicitly declared	☐
6	`meta.roles.approver` declared if approval gates exist	☐
7	`meta.roles.change-manager` declared if change-request exists	☐
8	All deployment steps wrapped in `ops.change-request`	☐
9	Every `ops.change-request` has a `rollback` specification	☐
10	Every `ops.manual` step has `requires_role`	☐
11	Approval gates exist before all production-modifying sequences	☐
12	Evidence is captured at key decision points (not just at the end)	☐
13	Output redaction covers all secrets that may appear in CLI output	☐
14	`meta.governance.allowed_commands` is restricted to needed tools	☐
15	Rollback runbook exists and has been tested in staging	☐

Chapter 10

Reference

This chapter provides complete field reference tables for all `gert.ops` types, role semantics, environment variables, and error codes. Use this chapter as a quick-lookup companion when authoring runbooks.

10.1 Runbook Metadata Fields

Field	Type	Required	Description
`apiVersion`	string	YES	Must be `runbook/v2`
`kit`	string	YES	Must be `ops/v1`
`meta.name`	string	YES	Unique runbook identifier
`meta.kind`	enum	YES	`change`, `incident`, `operational`
`meta.description`	string	YES	Human-readable description
`meta.roles.dri`	string	NO	DRI identity; defaults to run initiator
`meta.roles.approver`	string/list	NO	Approval gate signatories
`meta.roles.change-manager`	string	COND	Required if `ops.change-request` present
`meta.roles.responder`	string/list	COND	Required if `ops.incident` present
`meta.roles.incident-commander`	string	NO	Defaults to `meta.roles.dri`
`meta.roles.observer`	string/list	NO	Read-only run participants
`meta.change-id`	string	NO	External change ticket reference
`meta.sla.approval-timeout`	duration	NO	Gate approval timeout (e.g., `24h`)
`meta.sla.execution-timeout`	duration	NO	Total run time limit
`meta.sla.escalation-target`	string	NO	Identity notified on timeout
`meta.governance.allowed_commands`	list	NO	Step command allowlist
`meta.governance.denied_commands`	list	NO	Step command denylist
`meta.governance.deny_env_vars`	list	NO	Env var glob patterns to strip
`meta.governance.redact`	list	NO	Output redaction rules

10.2 `ops.cli` Field Reference

Field	Type	Required	Description
`id`	string	YES	Unique step identifier
`type`	string	YES	Must be `ops.cli`
`title`	string	YES	Human-readable step title
`command`	string	YES	Executable name (argv[0])
`args`	list	NO	Command arguments; templates supported
`allowed_commands`	list	NO	Step-level allowlist (intersects with runbook-level)
`redact`	list	NO	Step-level redaction rules
`capture`	map	NO	Maps variable names to output streams
`evidence.auto`	bool	NO	Auto-capture output as evidence
`evidence.label`	string	COND	Required when `evidence.auto: true`
`requires_role`	enum	NO	Role required to execute this step
`requires_any_role`	list	NO	Any of these roles required
`requires_all_roles`	list	NO	All of these roles required
`timeout`	duration	NO	Step execution timeout

10.3 `ops.manual` Field Reference

Field	Type	Required	Description
`id`	string	YES	Unique step identifier
`type`	string	YES	Must be `ops.manual`
`title`	string	YES	Human-readable step title
`requires_role`	enum	YES	Role required to execute
`instructions`	string	YES	Instructions shown to executor; templates supported
`evidence`	list	NO	Evidence specifications (see ğ10.7)
`requires_any_role`	list	NO	Any of these roles required
`requires_all_roles`	list	NO	All of these roles required
`timeout`	duration	NO	Step completion timeout

10.4 `ops.approval` Field Reference

Field	Type	Required	Description
`id`	string	YES	Unique step identifier
`type`	string	YES	Must be `ops.approval`
`title`	string	YES	Gate title shown to approvers
`description`	string	NO	Context provided to approvers; templates supported
`approvers`	list	NO	Overrides `meta.roles.approver`
`requires_any`	bool	NO	Any one approver sufficient (default: true)
`timeout`	duration	NO	Defaults to `meta.sla.approval-timeout`
`on_timeout`	enum	NO	`escalate` (default) or `abort`
`escalation_target`	string	NO	Overrides `meta.sla.escalation-target`
`evidence`	list	NO	Evidence required from approver

10.5 `ops.change-request` Field Reference

Field	Type	Required	Description
`id`	string	YES	Unique step identifier
`type`	string	YES	Must be `ops.change-request`
`title`	string	YES	Human-readable wrapper title
`change-id`	string	NO	External ticket reference; templates supported
`risk`	enum	NO	`low`, `medium`, `high`, `emergency`
`rollback`	object	NO	Rollback specification (strongly recommended)
`rollback.runbook`	string	COND	Relative path to rollback runbook
`rollback.inputs`	map	NO	Input bindings for rollback runbook
`steps`	list	YES	Change execution steps

10.6 `ops.incident` Field Reference

Field	Type	Required	Description
`id`	string	YES	Unique step identifier
`type`	string	YES	Must be `ops.incident`
`title`	string	YES	Incident title
`severity`	enum	YES	`sev1`, `sev2`, `sev3`, `sev4`
`incident-id`	string	NO	External ticket reference
`commander`	string	NO	Defaults to `meta.roles.incident-commander`
`sla.mitigation-timeout`	duration	NO	Time to first mitigation
`sla.resolution-timeout`	duration	NO	Total resolution time limit
`sla.escalation-target`	string	NO	Notified on SLA breach
`steps`	list	YES	Incident response steps

10.7 Evidence Type Field Reference

10.7.1 Common Fields (all evidence types)

Field	Type	Required	Description
`type`	enum	YES	Evidence type identifier
`label`	string	YES	Short identifier in evidence bundle
`required`	bool	NO	Step cannot complete without this (default: false)
`description`	string	NO	Instructions shown to submitter

10.7.2 `ops.evidence.screenshot`

No additional fields beyond the common set. The submitter uploads a binary file (PNG, JPEG, or PDF). Stored in `attachments/` under the run directory.

10.7.3 `ops.evidence.command-output`

Field	Type	Required	Description
`capture_from`	string	COND	Run variable to capture; required when not auto-captured

10.7.4 `ops.evidence.attestation`

Field	Type	Required	Description
`prompt`	string	YES	Statement the executor affirms verbatim

10.8 Role Semantics Reference

Role	Inherits From	Permissions
`dri`		Execute steps, submit evidence, approve gates, cancel run
`approver`		Sign `ops.approval` gates, view run status and evidence
`change-manager`		Approve/reject pre-execution gate, sign change record
`responder`		Execute `ops.manual` steps during `ops.incident` wrapper
`incident-commander`	`dri`	All DRI permissions plus IC-specific attestations and takeover
`observer`		View run status and evidence; no action permissions

10.9 Environment Variables

The following environment variables are recognized by `gert.ops` and affect runtime behavior:

Variable	Description
`GERT_OPS_ROLE`	Declares the current user's role for this run. Overrides `meta.roles.*` assignment. Useful in CI environments.
`GERT_OPS_CHANGE_ID`	Injects a change ticket ID without prompting. Used in automated pre-production pipelines.
`GERT_OPS_APPROVER_TIMEOUT`	Overrides `meta.sla.approval-timeout` for this run. Duration string.
`GERT_OPS_EVIDENCE_DIR`	Override the evidence output directory. Default: `.runbook/runs/<run-id>/attachments/`.
`GERT_OPS_FREEZE_CHECK_DISABLE`	Set to `1` to disable freeze window checks. Requires `risk: emergency` or superuser authority.
`GERT_OPS_BUNDLE_ON_COMPLETE`	Set to `1` to automatically write the evidence bundle JSON on run completion.

10.10 Error Codes

The following error codes are specific to `gert.ops`. Core gert error codes are documented in the *gert v2 Design Document*.

Code	Exit	Description
`OPS001`	1	Kit declaration missing: `kit: ops/v1` required for `ops.*` step types
`OPS002`	1	`ops.change-request` present but no `meta.roles.change-manager` declared
`OPS003`	1	`ops.incident` present but no `meta.roles.responder` declared
`OPS004`	1	`ops.manual` step has no `requires_role` declaration
`OPS010`	2	Role check failed: executing user does not hold required role
`OPS011`	2	Approval gate rejected by approver
`OPS012`	2	Approval gate timed out with `on_timeout: abort`
`OPS013`	2	Change-manager pre-execution approval rejected
`OPS014`	2	Freeze window active: change risk level `low/medium` blocked
`OPS020`	3	Required evidence not submitted before step completion
`OPS021`	3	Evidence submission by user without required role
`OPS030`	4	Rollback runbook not found at declared path
`OPS031`	4	Rollback runbook failed during automatic rollback injection
`OPS040`	5	Incident SLA resolution timeout: run suspended, IC acknowledgement required
`OPS041`	5	Incident commander takeover failed: declared commander not in `meta.roles`